

Kubernetes

What is Kubernetes?

Kubernetes (K8s) is an open-source **container orchestration platform**.

It helps **automate deployment, scaling, and management** of containerized applications (like Docker containers).

Think of it as the **“operating system for the cloud”** — it doesn’t run your app itself, but it tells your containers where to run, makes sure they stay running, and helps them handle traffic and failures.

Problem Before Kubernetes:

Imagine you built an app (say an **online food delivery app** 🍕). You put it inside a **Docker container** so it can run anywhere. That’s cool, but now problems start:

1. **Scaling:**

What if **10,000 people order pizza at once?**

One container can’t handle that — you need many copies of the app running.

2. **Load balancing:**

You now have 10 containers running — how do you make sure incoming food orders are distributed evenly?

3. **Failure handling:**

What if one container crashes? Who restarts it?

4. **Deployment updates:**

You want to release a new feature (e.g., “track your pizza delivery 🚲”) without stopping the app.

How do you update containers with **zero downtime?**

5. **Multi-server management:**

Your containers are spread across **many machines (servers)**. Managing them manually is like herding cats 🐱.

✅ How Kubernetes Solves These Problems

1. Scaling apps automatically

If traffic increases, Kubernetes creates more containers. If traffic decreases, it shuts down extra ones → saves money.

🔧 Example: During **lunch time**, it might run **50 containers**, but at night, only **5 containers**.

2. Load balancing

Kubernetes acts like a **traffic police** 🚦, sending user requests evenly to all containers so none gets overloaded.

3. Self-healing

If a container dies, Kubernetes **automatically restarts** it.

Analogy: Like a **factory manager** replacing a broken worker robot with a new one.

4. Rolling updates

Kubernetes updates containers one by one so the app **never goes offline**.

Analogy: Like changing airplane engines mid-flight ✈️ without landing.

5. Multi-machine orchestration

Kubernetes treats all your servers as **one big computer**. You don't care where a container runs — Kubernetes decides the best spot.

Analogy: Like a **hotel manager** assigning guests to rooms, making sure space is used efficiently.

🏠 Analogy: Restaurant Kitchen

Imagine your food delivery business 🍔:

- **Containers** = Chefs in the kitchen 👨🍳👩🍳.

Each chef (container) knows how to cook pizza.

- **Kubernetes** = Kitchen Manager 👨🍳.

The manager decides:

- How many chefs are needed right now?
- Which orders go to which chef?
- If a chef leaves, replace them with a new one.
- Train new chefs while old ones finish orders (rolling updates).
- Assign chefs to different kitchens if one kitchen is full.

Kubernetes = Container Orchestrator

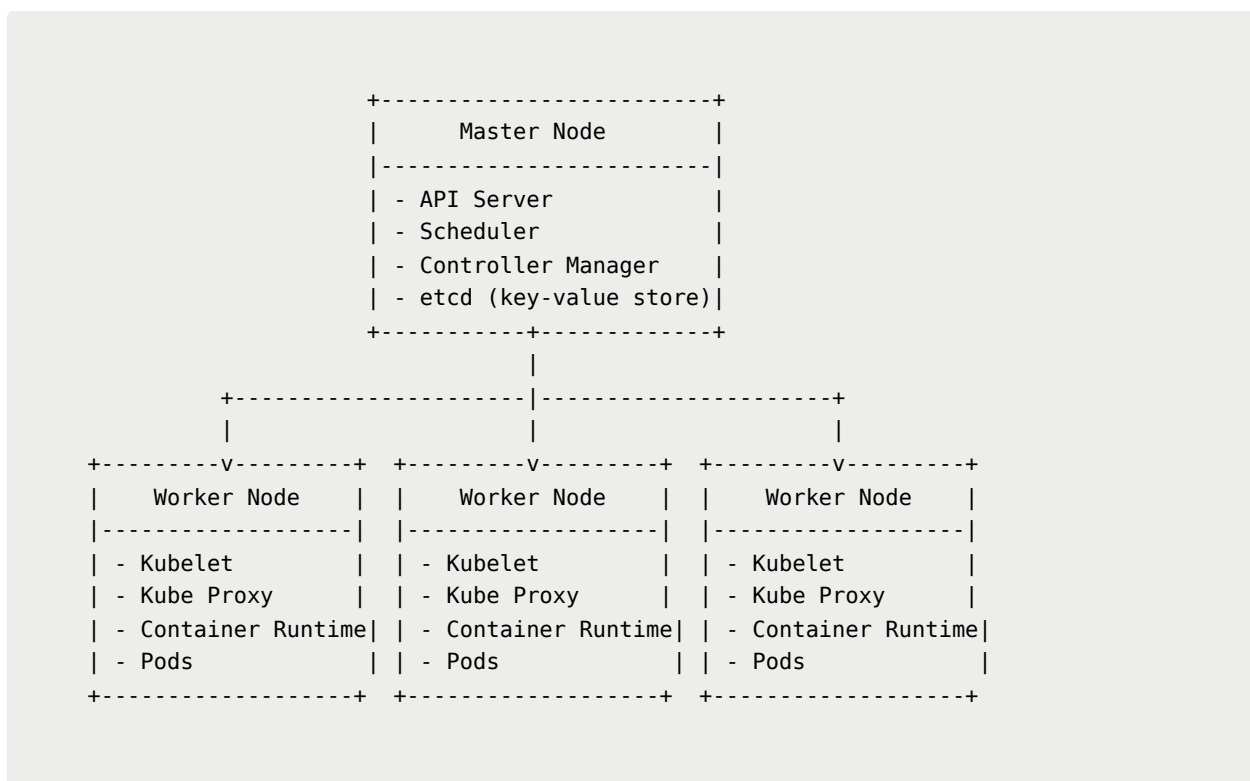
It solves the problems of **scaling, load balancing, self-healing, rolling updates, and multi-machine management** in cloud-native applications.

🧠 Basic Terminology (with analogies)

Term	Explanation	Analogy
Cluster	A set of machines (nodes) that run your applications.	A company with many computers (employees) working together.
Node	A single machine (VM or physical). Can be Master or Worker.	An employee in a company.
Pod	The smallest unit in Kubernetes. A pod contains one or more containers that run together.	A team (1-2 members) working on a task.
Container	Lightweight, isolated app environment (e.g., Docker).	An app running inside a portable box.
Deployment	Describes the desired state (e.g., 3 pods running your app). Kubernetes maintains this state.	Telling HR: "I need 3 people working on this task at all times."

Service	A stable endpoint (IP/port) to access pods. Handles networking/load balancing.	A receptionist who always connects you to the right team member.
Namespace	A way to divide cluster resources between multiple teams/projects.	Separate departments in a company (HR, IT, Marketing).
ReplicaSet	Ensures a specified number of pod replicas are running at all times.	A manager ensuring there are always 3 workers on a task.
ConfigMap / Secret	Store configuration and sensitive info (passwords, URLs).	A secure locker or file cabinet with instructions.
Volume	Used for persistent storage (outside container life).	A shared drive or hard disk.

Kubernetes Architecture Overview



Key Components Explained

Component	Role
-----------	------

API Server	Front door of the cluster. You (or <code>kubectl</code>) interact with this.
Scheduler	Decides which pod goes to which node.
Controller Manager	Maintains the desired state (replicas, rollout, etc.).
etcd	Key-value store that stores cluster state.
Kubelet	Agent running on each node that talks to API server and runs containers.
Kube Proxy	Manages networking (routing requests to the correct pod).
Container Runtime	The software that actually runs containers (e.g., Docker, containerd).

Kubernetes Architecture

1. Control Plane (Brain)

The **control plane** manages the whole cluster. It ensures apps are running as desired.

It has several key components:

♦ a) API Server (kube-apiserver)

- The **front door** of Kubernetes.
- All communication (kubectl, dashboards, other components) goes through the API Server.
- It exposes a **REST API**.

👉 Analogy: **Reception desk** where all requests are submitted.

♦ b) etcd

- A **key-value database** that stores the entire cluster state.
- Stores info like: Which pods are running? Which nodes exist? What's the desired replica count?

- Highly available & consistent.

👉 Analogy: **Cluster memory / brain storage** 🗂️.

♦ c) Scheduler (kube-scheduler)

- Decides **which node** should run a newly created pod.
- Looks at resource requirements (CPU, RAM), policies, taints, affinity rules.

👉 Analogy: **HR manager** assigning workers to different offices based on skills and space.

♦ d) Controller Manager (kube-controller-manager)

- Runs background controllers that ensure the cluster state matches the desired state.
- Examples:
 - **ReplicaSet Controller** → Makes sure correct number of pods are running.
 - **Node Controller** → Monitors node health.
 - **Endpoint Controller** → Updates services with pod IPs.

👉 Analogy: **Supervisors** checking that the team works as planned.

♦ e) Cloud Controller Manager

- Integrates Kubernetes with cloud providers (AWS, GCP, Azure).
- Manages load balancers, storage volumes, etc.

👉 Analogy: **External supplier manager** who talks to cloud vendors.

2. Worker Nodes (Body 💪)

Each **worker node** runs your application containers.

It has components to communicate with the control plane:

♦ a) Kubelet

- An agent running on each node.
- Talks to API Server → receives pod specs → ensures containers are running.
- Reports back pod/node status.

👉 Analogy: **Site manager** at each office who follows instructions from HQ.

♦ b) Kube Proxy

- Handles networking rules.
- Ensures services can reach pods across nodes (load balancing, routing).

👉 Analogy: **Network admin** who manages the internal switchboard.

♦ c) Container Runtime

- The software that runs containers (Docker, containerd, CRI-O).
- Kubernetes itself does not run containers, it relies on runtime.

👉 Analogy: **Actual cooking stove** 🔥 where the meals (containers) are prepared.

3. Kubernetes Objects

On top of this architecture, you define objects (via YAML or kubectl):

- **Pod** = Smallest deployable unit (1 or more containers).
- **Service** = Stable network endpoint to reach pods.
- **Deployment** = Manages replica sets & rolling updates.
- **ConfigMap/Secret** = Config and sensitive info.
- **Ingress** = Routes external traffic into the cluster.

👉 These are like **blueprints** the control plane executes.



Workflow (How It All Works Together)

1. You apply a **Deployment YAML** (e.g., 3 replicas of nginx).

→ Sent to API Server.

2. **API Server stores desired state in etcd.**
 3. **Scheduler assigns pods to nodes** based on resources.
 4. **Controller Manager ensures replica count is maintained.**
 5. **Kubelet on each node talks to container runtime** → starts the pods.
 6. **Kube Proxy sets up networking** so traffic reaches the pods.
 7. If a pod fails → **Controller Manager + Kubelet restart it.**
-



Analogy: Big Restaurant Chain

- **Control Plane = Headquarters** (decides what each branch should do).
 - API Server = Reception desk 📞
 - etcd = Master files cabinet 📁
 - Scheduler = HR manager assigning chefs 👨🍳
 - Controller Manager = Supervisors making sure branches follow rules
 - **Worker Nodes = Branch Restaurants** 🍴
 - Kubelet = Branch Manager
 - Kube Proxy = Network admin, ensures orders flow correctly
 - Container Runtime = Kitchen stove, actually cooks
-

⚡ In short:

Kubernetes = **a distributed brain + body system.**

- **Brain (Control Plane):** Decides what should happen.
- **Body (Worker Nodes):** Executes and runs the workloads.
- **etcd:** Memory of the whole system.

Real Life Analogy

Imagine you're running a restaurant chain 🍔:

- **Master Node** = Manager who takes orders and assigns tasks.
- **Worker Node** = Cooks who prepare the dishes.
- **Pods** = Stations where food is prepared (burger station, fries station).
- **Services** = Waiters who serve customers, always pointing to the correct station.
- **Deployment** = You instruct that there must always be 2 burger stations open.

Why Use Kubernetes?

- ✓ Auto-healing (restarts crashed pods)
- ✓ Load balancing & service discovery
- ✓ Easy scaling (1 to 100 pods in seconds)
- ✓ Rollbacks & rolling updates
- ✓ Declarative infrastructure (YAML-based)

1. Basic Cluster Commands

Command	Description
<code>kubectl version</code>	Show client and server version
<code>kubectl cluster-info</code>	Display cluster info
<code>kubectl config view</code>	Show current kubeconfig
<code>kubectl get nodes</code>	List all cluster nodes
<code>kubectl describe node <node-name></code>	Detailed info about a node

2. Working with Pods

Command	Example	Description
<code>kubectl get pods</code>	List all pods in current namespace	
<code>kubectl get pods -A</code>	List all pods across all namespaces	
<code>kubectl describe pod mypod</code>	Detailed info about a pod	
<code>kubectl logs mypod</code>	Get logs from a pod	
<code>kubectl logs -f mypod</code>	Follow pod logs	
<code>kubectl exec -it mypod -- bash</code>	SSH into a pod (if shell exists)	
<code>kubectl delete pod mypod</code>	Delete a pod manually	

3. Deployments & ReplicaSets

Command	Example	Description
<code>kubectl create deployment nginx --image=nginx</code>	Create a deployment	
<code>kubectl get deployments</code>	List deployments	
<code>kubectl describe deployment nginx</code>	Deployment details	
<code>kubectl scale deployment nginx --replicas=3</code>	Scale a deployment	
<code>kubectl delete deployment nginx</code>	Delete a deployment	

4. Services

Command	Example	Description
<code>kubectl expose deployment nginx --port=80 --type=NodePort</code>	Expose deployment as a service	
<code>kubectl get svc</code>	List all services	
<code>kubectl describe svc nginx</code>	Service details	
<code>kubectl delete svc nginx</code>	Delete a service	

5. Apply/Manage YAML Files

Command	Example	Description
<code>kubectl apply -f myapp.yaml</code>	Apply/create resource from YAML	
<code>kubectl create -f myapp.yaml</code>	Same as above (older style)	
<code>kubectl delete -f myapp.yaml</code>	Delete resources defined in YAML	
<code>kubectl diff -f myapp.yaml</code>	See diff before applying	
<code>kubectl get -f myapp.yaml</code>	Get resource defined in YAML	

6. ConfigMaps & Secrets

Command	Example	Description
<code>kubectl create configmap myconfig --from-literal=key1=value1</code>	Create ConfigMap from CLI	
<code>kubectl create secret generic mysecret --from-literal=password=1234</code>	Create secret	
<code>kubectl get configmap</code>	List configmaps	
<code>kubectl get secret</code>	List secrets	
<code>kubectl describe configmap myconfig</code>	ConfigMap details	
<code>kubectl describe secret mysecret</code>	Secret details (base64 encoded)	

7. Namespaces

Command	Example	Description
<code>kubectl get namespaces</code>	List all namespaces	
<code>kubectl create namespace dev</code>	Create a new namespace	
<code>kubectl delete namespace dev</code>	Delete a namespace	
<code>kubectl get pods -n dev</code>	Get pods in specific namespace	
<code>kubectl config set-context --current --namespace=dev</code>	Set default namespace	

kubectl config set-context --current --namespace=<namespace-name>		
---	--	--

8. Rollouts & Updates

Command	Example	Description
<code>kubectl rollout status deployment nginx</code>	Check rollout status	
<code>kubectl rollout restart deployment nginx</code>	Restart a deployment	
<code>kubectl rollout undo deployment nginx</code>	Rollback deployment	

9. Monitoring & Debugging

Command	Example	Description
<code>kubectl top nodes</code>	CPU/memory usage of nodes	
<code>kubectl top pods</code>	CPU/memory usage of pods	
<code>kubectl describe <resource> <name></code>	Debug any resource	
<code>kubectl events</code> (with plugin)	View recent cluster events	

10. Clean Up & Utilities

Command	Example	Description
<code>kubectl delete all --all</code>	Delete all resources in namespace	
<code>kubectl get all</code>	Get all resources (pods, services, deployments)	
<code>kubectl explain pod</code>	Show detailed structure and docs for a resource	
<code>kubectl port-forward pod/mypod 8080:80</code>	Access pod locally on port 8080	

Bonus: YAML Generator Shortcut

bash

CopyEdit

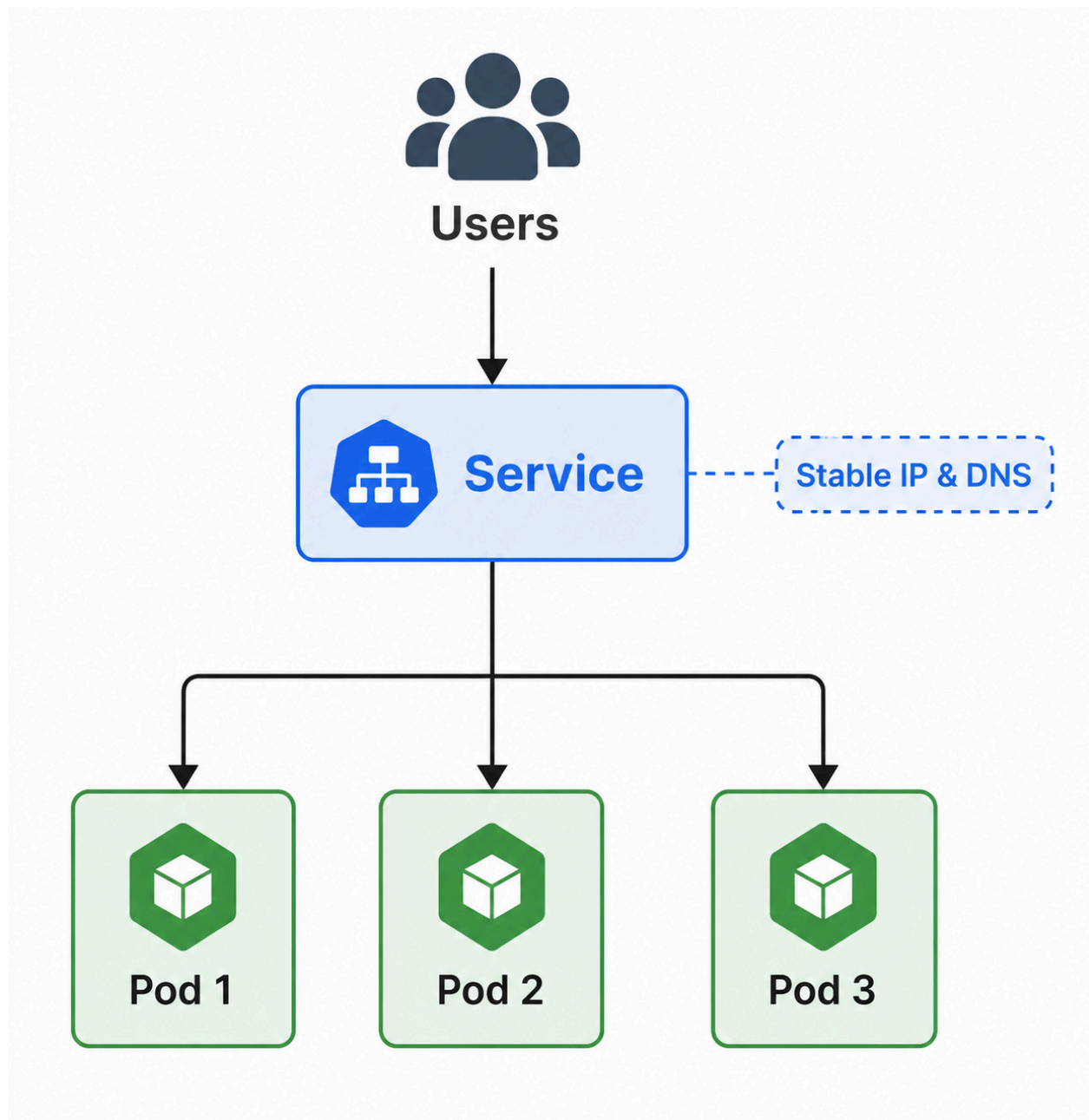
```
kubectl create deployment myapp --image=myimage --dry-run=client -o yaml > myapp.yaml
```

 Creates a YAML file you can edit and apply!

Want this list as a **cheat sheet PDF**, or want to practice these commands in a Minikube or Docker Desktop setup? I can guide you step-by-step.

K8s services in detail

A **Service** is a Kubernetes object that provides a **permanent network endpoint** for accessing one or more Pods.



The service sits in front of Pods and forwards traffic.

Why do we need a Service?

Suppose your deployment has 3 Pods.

```
Spring Boot Deployment
```

```
Pod A -> 10.1.1.2  
Pod B -> 10.1.1.5  
Pod C -> 10.1.1.8
```

Pods get IPs dynamically.

Now imagine Pod B crashes.

Kubernetes creates a new Pod.

```
Old IP  
10.1.1.5 ✖  
  
New Pod  
10.1.1.10
```

If another application was calling

```
http://10.1.1.5:8080
```

it breaks.

That's why we use a Service.

The Service always has the same IP and DNS.

Example:

```
http://user-service:8080
```

Even if Pods change, the Service knows where to send requests.

How Service finds Pods?

Services use **Labels**.

Pods

```
labels:
```

```
app: user-service
```

Service

```
selector:  
  app: user-service
```

If labels match,

Service connects automatically.

```
Service  
  selector:  
    app=user-service
```

|



```
Pod  
app=user-service ✓
```

Types of Service

1. ClusterIP (Default)

Most commonly used.

Only accessible **inside the Kubernetes cluster**.

Cluster

Frontend

|



User-Service

|



Pods

Outside users cannot access it.

Perfect for

- Database
- Redis
- Kafka
- Internal microservices
- Payment service
- Inventory service

Example

```
type: ClusterIP
```

2. NodePort

Exposes the Service on every Kubernetes node using a high port (default range **30000–32767**).

Internet

|

192.168.1.20:30080

|

Node

|

Service

|

Pods

If your node IP is

192.168.1.20

and NodePort is

30080

You access

http://192.168.1.20:30080

Example

```
type: NodePort

ports:
- port: 80
  targetPort: 8080
  nodePort: 30080
```

Use cases:

- Local development
- Testing
- Small labs

Not recommended for production internet traffic.

3. LoadBalancer

Used mainly in cloud environments like AWS, Azure, or GCP.

Internet

|

Cloud Load Balancer

|

Service

|

Pods

Cloud provider automatically creates an external load balancer.

Example

```
type: LoadBalancer
```

Users access the external IP provided by the cloud.

Great for production APIs.

4. ExternalName

Does not forward traffic to Pods.

Instead, it maps a Kubernetes Service name to an external DNS name.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: database.example.com
```

Now applications inside the cluster can connect to:

```
external-db
```

which resolves to:

```
database.example.com
```

Useful for integrating external databases or third-party services

Creating Deployment and service of nginx server

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

```

---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP

```

r

To access on local machine use port forwarding because we are using service type ClusterIP which for internal cluster.

```
kubectl port-forward svc/nginx-service 8080:80
```

Working with Persistent Volume

A **Kubernetes Volume** is a storage location that allows containers to **store data**.

Without a volume, data is **lost when a Pod is deleted or restarted**.

Without Volume

```

Pod
├── Container
│   └── /data

```

If the Pod dies:

Pod ❌

Data ❌ Lost

With Volume

```
Pod
├── Container
│   └── /data
└── Volume
```

If the container restarts:

Container Restart

Data ✅ Available

Types of Kubernetes Volumes

1. emptyDir

What is it?

An **empty directory** created when the Pod starts.

- Created with the Pod
- Shared by all containers inside the same Pod
- Deleted when the Pod is deleted

Architecture

```
Pod
├── Container A
```

```
└─ Container B
└─ emptyDir
```

Example

```
apiVersion: v1
kind: Pod
metadata:
  name: emptydir-demo

spec:
  containers:
    - name: app
      image: nginx

      volumeMounts:
        - name: cache
          mountPath: /cache

  volumes:
    - name: cache
      emptyDir: {}
```

Everything stored inside `/cache` disappears when the Pod is deleted.

Use Cases

- Temporary cache
- Scratch files
- Sharing files between containers

2. hostPath

What is it?

Mounts a folder from the Kubernetes Node into the Pod.

```
Node
└─ /var/log
```

|
▼
Pod

Example

```
volumes:  
- name: logs  
  hostPath:  
    path: /var/log
```

Mount

```
volumeMounts:  
- name: logs  
  mountPath: /logs
```

Now

```
Node:  
/var/log  
  
==>  
  
Container:  
/logs
```

Advantages

- Access node files
- Useful for monitoring

Disadvantages

If Pod moves to another node

```
Node1 -> Has Data
```

```
Node2 -> No Data
```

Hence **not recommended for production applications**.

Real Example

Prometheus

Grafana Agent

Fluentd

These tools often read node logs using `hostPath`.

3. Persistent Volume (PV)

Problem

Suppose MySQL is running.

```
Pod
```

```
MySQL Database
```

Pod crashes.

Everything disappears.

Need permanent storage.

That's where PV comes.

What is PV?

A **PersistentVolume** is the actual storage available in the cluster.

Storage may come from

- AWS EBS

- Azure Disk
- Google Persistent Disk
- NFS
- Local Disk

Think of PV as a **hard disk**.

```
Storage  
  
100 GB SSD  
  
↓  
  
Persistent Volume
```

Example

```
apiVersion: v1  
kind: PersistentVolume  
  
spec:  
  capacity:  
    storage: 20Gi  
  
  accessModes:  
    - ReadWriteOnce
```

4. Persistent Volume Claim (PVC)

Pods never directly use a PV.

Instead they ask Kubernetes.

| "Please give me 10GB storage."

This request is PVC.

```
Pod
```

↓
PVC
↓
PV
↓
Disk

Example

```
apiVersion: v1
kind: PersistentVolumeClaim

spec:

  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 10Gi
```

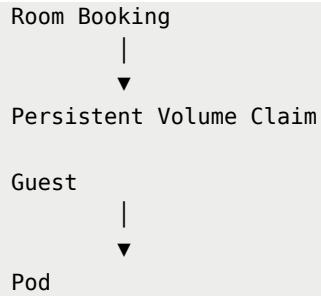
Mount

```
volumes:
- name: mysql-storage
  persistentVolumeClaim:
    claimName: mysql-pvc
```

PV vs PVC

Imagine renting a hotel room.

Hotel Building
|
▼
Persistent Volume



PV = Hotel Room

PVC = Booking

Pod = Guest

5. ConfigMap Volume

Applications need configuration.

Example

```
application.yml

server.port=8080

spring.datasource.url=...
```

Instead of putting config inside Docker image,
Store it separately.

```
ConfigMap

↓

Pod

↓

/config/application.yml
```

Example

```
volumes:
- name: config-volume
  configMap:
    name: app-config
```

Mount

```
volumeMounts:
- name: config-volume
  mountPath: /config
```

Real Use

Spring Boot

```
application.yml
application-prod.yml
logback.xml
```

6. Secret Volume

Similar to ConfigMap

But stores sensitive data.

Example

- Password
- JWT Secret
- API Key
- SSL Certificate

Example

```
volumes:
- name: secret-volume
  secret:
    secretName: mysql-secret
```

Mounted as

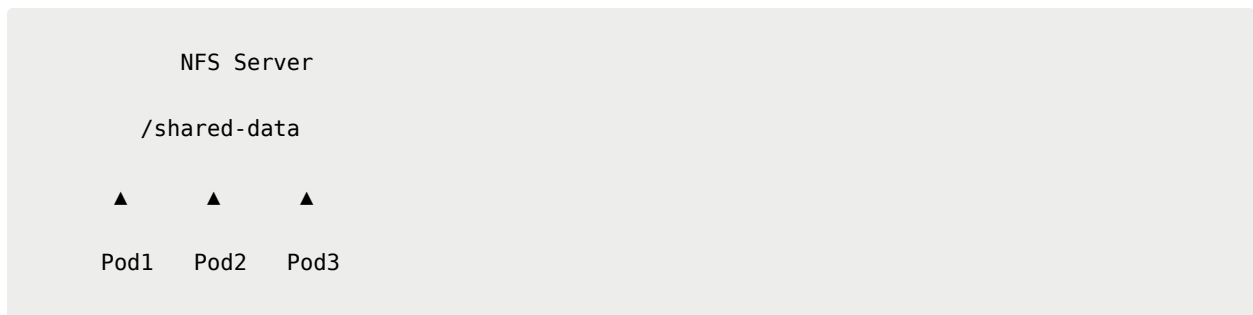
```
/secrets/password
/secrets/username
```

Spring Boot can read these files at startup.

7. NFS Volume

NFS = Network File System

Many Pods can use the same storage.



Example

```
volumes:
- name: shared-storage

  nfs:
    server: 192.168.1.100
    path: /shared
```

Use Cases

- Shared uploads
 - Media files
 - Multiple replicas reading the same files
-

8. Cloud Provider Volumes

Production Kubernetes clusters usually use cloud-managed storage.

Examples

AWS

Pod

↓

PVC

↓

AWS EBS

↓

SSD

Azure

Azure Disk

Google Cloud

Persistent Disk

These are dynamically created by Kubernetes using a **StorageClass**.

StorageClass (Very Important)

Instead of manually creating PVs,
Modern Kubernetes uses **StorageClass**.

Old Flow

```
graph TD; Admin --> CreatePV[Create PV]; CreatePV --> Developer; Developer --> PVC
```

Admin
↓
Create PV
↓
Developer
↓
PVC

Modern Flow

```
graph TD; Developer --> PVC; PVC --> StorageClass; StorageClass --> AutomaticPV[Automatic PV]; AutomaticPV --> CloudDisk[Cloud Disk]
```

Developer
↓
PVC
↓
StorageClass
↓
Automatic PV
↓
Cloud Disk

This is called **Dynamic Provisioning**.

Example

```
storageClassName: standard
```

Access Modes

ReadWriteOnce (RWO)

Only one node can mount the volume for read/write.

Node1 ✓

Node2 ✗

Used by MySQL.

ReadOnlyMany (ROX)

Many Pods can read.

No writing.

Pod1 Read ✓

Pod2 Read ✓

Write ✗

ReadWriteMany (RWX)

Everyone can read and write.

Pod1 ✓

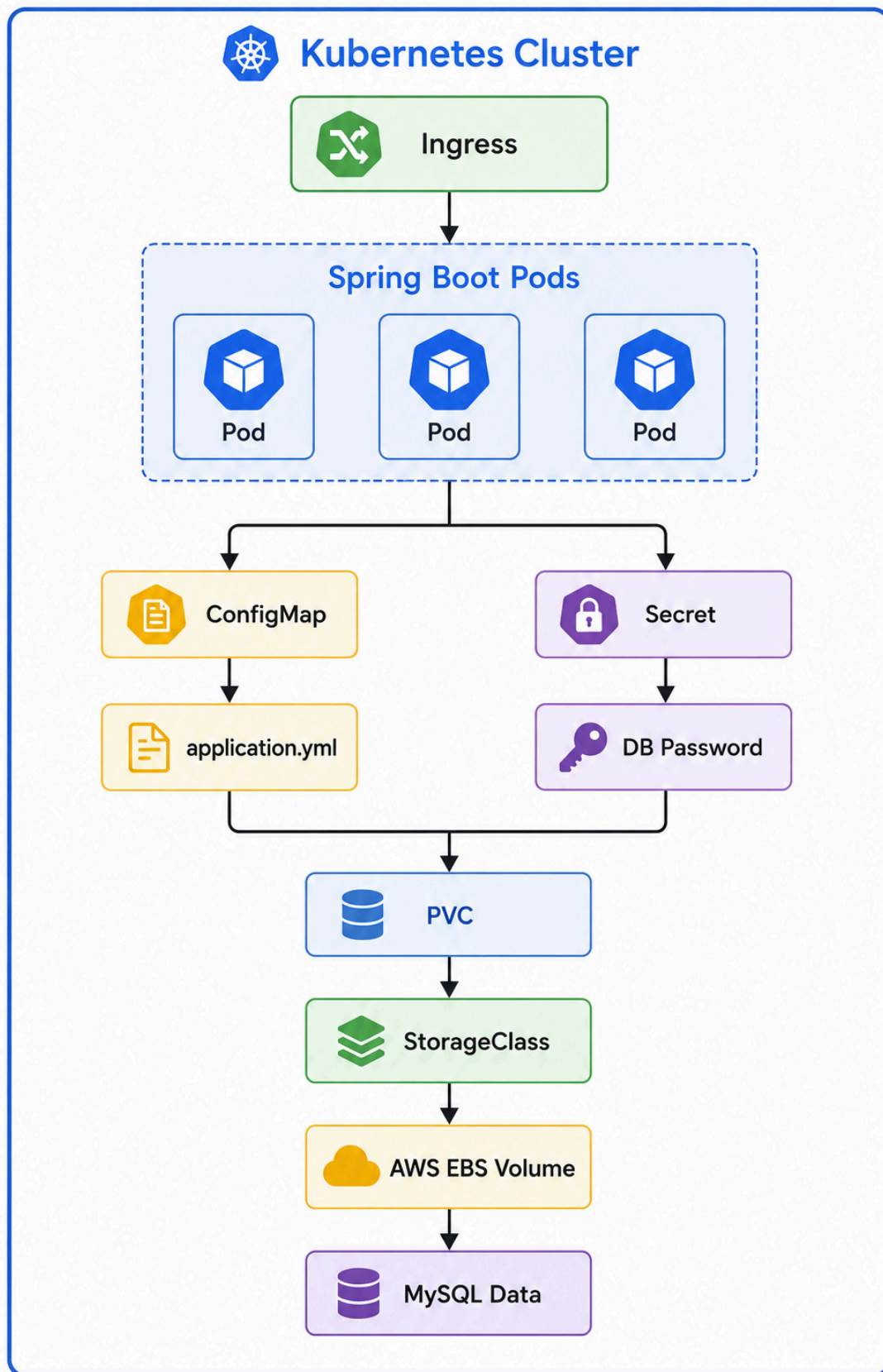
Pod2 ✓

Pod3 ✓

Mostly used with

- NFS
 - CephFS
 - Amazon EFS
 - Azure Files
-

Real Production Example



Below is a **production-style MySQL deployment** in Kubernetes using:

-  PersistentVolume
-  PersistentVolumeClaim
-  Secret (for password)
-  Deployment
-  ClusterIP Service

This is suitable for learning and small projects.

1. Secret (mysql-secret.yaml)

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque

data:
  MYSQL_ROOT_PASSWORD: cm9vdDEyMw==# root123
  MYSQL_DATABASE: YmxvZ19hcHA=# blog_app
```

| Encode values using:

```
echo -n "root123" | base64 echo -n "blog_app" | base64
```

2. PersistentVolume (pv.yaml)

| For local clusters like Docker Desktop, Minikube, or Kind.

```
apiVersion: v1
kind: PersistentVolume

metadata:
  name: mysql-pv
```

```
spec:
  capacity:
    storage: 5Gi

  accessModes:
    - ReadWriteOnce

  persistentVolumeReclaimPolicy: Retain

  hostPath:
    path: /mnt/mysql-data
```

3. PersistentVolumeClaim (pvc.yaml)

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: mysql-pvc

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 5Gi
```

4. MySQL Deployment (mysql-deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: mysql

spec:
  replicas: 1
```

```

selector:
  matchLabels:
    app: mysql

template:

  metadata:
    labels:
      app: mysql

  spec:
    containers:
      - name: mysql

        image: mysql:8.4

        ports:
          - containerPort: 3306

        env:
          - name: MYSQL_ROOT_PASSWORD
            valueFrom:
              secretKeyRef:
                name: mysql-secret
                key: MYSQL_ROOT_PASSWORD

          - name: MYSQL_DATABASE
            valueFrom:
              secretKeyRef:
                name: mysql-secret
                key: MYSQL_DATABASE

        volumeMounts:
          - name: mysql-storage
            mountPath: /var/lib/mysql

    volumes:
      - name: mysql-storage

        persistentVolumeClaim:
          claimName: mysql-pvc

```

5. Service (mysql-service.yaml)

```

apiVersion: v1
kind: Service

```

```
metadata:
  name: mysql

spec:
  type: ClusterIP

  selector:
    app: mysql

  ports:
    - port: 3306
      targetPort: 3306
```

Apply Everything

```
kubectl apply-f mysql-secret.yaml
kubectl apply-f pv.yaml
kubectl apply-f pvc.yaml
kubectl apply-f mysql-deployment.yaml
kubectl apply-f mysql-service.yaml
```

Or place all resources into one file and run:

```
kubectl apply-f mysql.yaml
```

Verify

Check Pods:

```
kubectl get pods
```

Check PVC:

```
kubectl get pvc
```

Example:

NAME	STATUS	VOLUME	CAPACITY
mysql-pvc	Bound	mysql-pv	5Gi

Check PV:

```
kubectl get pv
```

Check Service:

```
kubectl get svc
```

Connect from Spring Boot

Since the MySQL Service is named **mysql**, other Pods in the same namespace can connect using:

```
spring.datasource.url=jdbc:mysql://mysql:3306/blog_app
spring.datasource.username=root
spring.datasource.password=root123
```

Kubernetes DNS automatically resolves `mysql` to the Service IP.

Production Improvements

For production deployments, it's common to replace the `hostPath` PV with a `StorageClass` and use a `StatefulSet` instead of a `Deployment` for MySQL.

Typical production architecture:

Spring Boot Pods
|
▼

```
mysql.default.svc.cluster.local
|
ClusterIP Service
|
MySQL StatefulSet
|
PersistentVolumeClaim (PVC)
|
StorageClass (gp3, standard, etc.)
|
AWS EBS / Azure Disk / GCP Persistent Disk
```

This provides stable storage, predictable Pod identities, and is the recommended pattern for stateful databases in Kubernetes.

Mysql with volume setup

- **Phase 1 (Local)** → Deployment + PV + PVC + Service (easy to understand)
- **Phase 2 (Production)** → StatefulSet + StorageClass + PVC (real-world)

Let's build the **local version first**, then I'll explain exactly what changes in production.

Project Structure

```
mysql-k8s/
|
├─ 01-secret.yaml
├─ 02-pv.yaml
├─ 03-pvc.yaml
├─ 04-deployment.yaml
└─ 05-service.yaml
```

Step 1 : Secret

01-secret.yaml

```

apiVersion: v1
kind: Secret

metadata:
  name: mysql-secret

type: Opaque

stringData:
  MYSQL_ROOT_PASSWORD: root123
  MYSQL_DATABASE: blog_app
  MYSQL_USER: durgesh
  MYSQL_PASSWORD: durgesh123

```

💡 `stringData` is easier for learning because Kubernetes automatically Base64-encodes it. In production, many teams still use `stringData` in GitOps workflows or use external secret managers.

Step 2 : Persistent Volume

02-pv.yaml

```

apiVersion: v1
kind: PersistentVolume

metadata:
  name: mysql-pv

spec:

  capacity:
    storage: 5Gi

  accessModes:
    - ReadWriteOnce

  persistentVolumeReclaimPolicy: Retain

  hostPath:
    path: /mnt/mysql-data

```

What is `/mnt/mysql-data` ?

When MySQL stores data:

```
/var/lib/mysql
```

inside the container,
Kubernetes maps it to

```
Node
```

```
/mnt/mysql-data
```

All database files live there.

Step 3 : PVC

03-pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: mysql-pvc

spec:

  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 5Gi
```

Step 4 : Deployment

04-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: mysql

spec:

  replicas: 1

  selector:
    matchLabels:
      app: mysql

  template:

    metadata:
      labels:
        app: mysql

    spec:

      containers:

        - name: mysql

          image: mysql:8.4

          ports:
            - containerPort: 3306

          env:

            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_ROOT_PASSWORD

            - name: MYSQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_DATABASE

            - name: MYSQL_USER
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: MYSQL_USER
```

```
- name: MYSQL_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mysql-secret
      key: MYSQL_PASSWORD

volumeMounts:

- name: mysql-storage
  mountPath: /var/lib/mysql

volumes:

- name: mysql-storage

  persistentVolumeClaim:
    claimName: mysql-pvc
```

Step 5 : Service

05-service.yaml

```
apiVersion: v1
kind: Service

metadata:
  name: mysql

spec:

  type: ClusterIP

  selector:
    app: mysql

  ports:

    - port: 3306
      targetPort: 3306
```

Apply Everything

```
kubectl apply -f .
```

Verify

```
kubectl get all
```

```
pod/mysql  
deployment/mysql  
service/mysql
```

PVC

```
kubectl get pvc
```

```
mysql-pvc  
STATUS  
Bound
```

PV

```
kubectl get pv
```

```
mysql-pv  
STATUS  
Bound
```

What Changes in Production?

Only a few files change.

Remove PV

Delete

```
02-pv.yaml
```

because the cloud creates the PV automatically.

Add StorageClass

Example (AWS gp3):

```
apiVersion: storage.k8s.io/v1
kind: StorageClass

metadata:
  name: gp3

provisioner: ebs.csi.aws.com

parameters:
  type: gp3

volumeBindingMode: WaitForFirstConsumer
```

PVC Changes

Old

```
resources:
  requests:
    storage: 5Gi
```

Production

```
storageClassName: gp3
```

```
resources:  
  requests:  
    storage: 20Gi
```

PVC automatically creates

AWS EBS Disk

↓

Persistent Volume

Deployment Changes

Replace

```
kind: Deployment
```

with

```
kind: StatefulSet
```

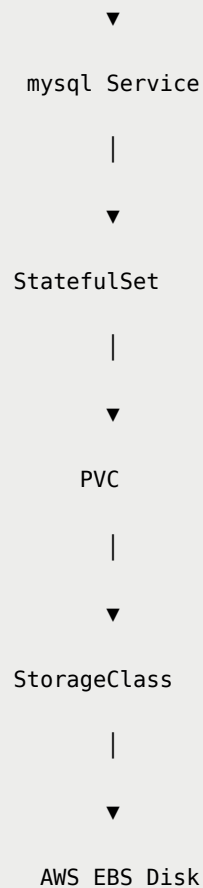
because databases need:

- Stable Pod names (`mysql-0` , `mysql-1`)
- Stable network identity
- Stable storage association

Production Architecture

Spring Boot

|



Deploying Spring Boot:

1. **Dockerize your Spring Boot app**
2. **Create Deployment YAML**
3. **Create Service YAML**
4. **Apply to Kubernetes**

Step 1: Dockerize Your Spring Boot App

Make sure you have a `Dockerfile` like this:

```
# Use a base image with Java
FROM openjdk:17-jdk-slim

# Copy your JAR file into the container
COPY target/myapp.jar app.jar

# Run the app
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Then build and push the Docker image:

```
docker build -t your-dockerhub-username/springboot-app:latest .
docker push your-dockerhub-username/springboot-app:latest
```

Step 2: Deployment YAML (`springboot-deployment.yaml`)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot-app
  template:
    metadata:
      labels:
```

```
    app: springboot-app
spec:
  containers:
  - name: springboot-app
    image: your-dockerhub-username/springboot-app:latest
    ports:
    - containerPort: 8080
```

Step 3: Service YAML (`springboot-service.yaml`)

```
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  selector:
    app: springboot-app
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
  type: LoadBalancer # or NodePort if you're on Minikube
```

Step 4: Apply to Kubernetes

```
kubectl apply -f springboot-deployment.yaml
kubectl apply -f springboot-service.yaml
```

Check everything is running:

```
kubectl get pods
kubectl get svc
```



Accessing the App

- If you're using Minikube:

```
minikube service springboot-service
```

- If on Cloud (EKS, GKE, etc):

- Use the **External IP** shown in `kubectl get svc`

Simplest working Kubernetes Deployment + Service for your image batchlcwd/docker-example-image:v1.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-sample
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot-sample
  template:
    metadata:
```

```

    labels:
      app: springboot-sample
  spec:
    containers:
      - name: springboot-app
        image: batchlcwd/docker-example-image:v1
        ports:
          - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  selector:
    app: springboot-sample
  ports:
    - port: 8080          # service port
      targetPort: 8080    # container port
      nodePort: 30080
  type: NodePort          # change to ClusterIP if only internal
                           # access

```

Apply them:

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml

```

```

kubectl get pods
kubectl get svc

```

If you used NodePort, Kubernetes will assign a random port (e.g. 31234). You can then access:

```
http://<node-ip>:<node-port>/
```

Create a ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: springboot-datasource-config
data:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/mydb
  SPRING_DATASOURCE_USERNAME: myuser
  SPRING_DATASOURCE_PASSWORD: mypassword
```

Update your Deployment to inject these values as environment variables:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-sample
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot-sample
  template:
    metadata:
```

```

labels:
  app: springboot-sample
spec:
  containers:
  - name: springboot-app
    image: batchlcwd/docker-example-image:v1
    ports:
    - containerPort: 8080
    envFrom:
    - configMapRef:
        name: springboot-datasource-config

```

```

kubectl apply -f springboot-datasource-config.yaml
kubectl apply -f springboot-deploy.yaml

```

Deploying with mysql and spring boot

```

# Simple MySQL + Spring Boot (single-file, minimal)
---
apiVersion: v1
kind: Service
metadata:
  name: mysql
spec:
  selector:
    app: mysql
  ports:
  - port: 3306
    targetPort: 3306

```

```

    type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
          - name: MYSQL_ROOT_PASSWORD
            value: rootpassword
          - name: MYSQL_DATABASE
            value: mydb
          - name: MYSQL_USER
            value: myuser
          - name: MYSQL_PASSWORD
            value: mypassword
        ports:
          - containerPort: 3306
        volumeMounts:
          - name: mysql-data
            mountPath: /var/lib/mysql
      volumes:
      - name: mysql-data
        emptyDir: {}

```

```

---
apiVersion: v1
kind: Service
metadata:
  name: springboot-service
spec:
  selector:
    app: springboot-sample
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
    type: NodePort
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-sample
spec:
  replicas: 1
  selector:
    matchLabels:
      app: springboot-sample
  template:
    metadata:
      labels:
        app: springboot-sample
    spec:
      containers:
        - name: springboot-app
          image: batchlcwd/docker-example-image:v1
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATASOURCE_URL
              value: "jdbc:mysql://mysql:3306/mydb?useSSL=fal

```

```
se&allowPublicKeyRetrieval=true"
- name: SPRING_DATASOURCE_USERNAME
  value: "myuser"
- name: SPRING_DATASOURCE_PASSWORD
  value: "mypassword"
```

```
kubectl apply -f simple-spring-mysql.yml
```

```
kubectl get pods
kubectl get svc
```

```
http://<node-ip>:30080/
```

Microservices

```
# Namespace (optional but recommended)
apiVersion: v1
kind: Namespace
metadata:
  name: quiz-app

---

# Secret for sensitive values (example for config-server Git)
apiVersion: v1
kind: Secret
metadata:
  name: config-git-credentials
```

```

    namespace: quiz-app
  type: Opaque
  stringData:
    GIT_USERNAME: your-username
    GIT_PASSWORD: your-token-or-password
---
# CONFIG-SERVER
apiVersion: v1
kind: ConfigMap
metadata:
  name: config-server-config
  namespace: quiz-app
data:
  SPRING_APPLICATION_NAME: config-server
  SERVER_PORT: "8888"
  # Spring Cloud Config (Git backend example)
  SPRING_PROFILES_ACTIVE: git
  SPRING_CLOUD_CONFIG_SERVER_GIT_URI: https://github.com/your
-org/your-config-repo.git
  SPRING_CLOUD_CONFIG_SERVER_GIT_DEFAULT_LABEL: main
  # Optional: local clone perf
  SPRING_CLOUD_CONFIG_SERVER_GIT_CLONE_ON_START: "true"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: config-server
  namespace: quiz-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: config-server
  template:

```

```

    metadata:
      labels:
        app: config-server
    spec:
      containers:
        - name: config-server
          image: your-registry/config-server:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8888
          envFrom:
            - configMapRef:
                name: config-server-config
            - secretRef:
                name: config-git-credentials
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8888
              initialDelaySeconds: 10
              periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8888
              initialDelaySeconds: 20
              periodSeconds: 20
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: config-server
      namespace: quiz-app
    spec:
      selector:

```

```

    app: config-server
  ports:
    - port: 8888
      targetPort: 8888

---
# SERVICE-REGISTRY (Eureka Server)
apiVersion: v1
kind: ConfigMap
metadata:
  name: service-registry-config
  namespace: quiz-app
data:
  SPRING_APPLICATION_NAME: service-registry
  SERVER_PORT: "8761"
  EUREKA_CLIENT_REGISTER_WITH_EUREKA: "false"
  EUREKA_CLIENT_FETCH_REGISTRY: "false"
  EUREKA_SERVER_ENABLE_SELF_PRESERVATION: "false"

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: service-registry
  namespace: quiz-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: service-registry
  template:
    metadata:
      labels:
        app: service-registry
    spec:
      containers:

```

```

- name: service-registry
  image: your-registry/service-registry:latest
  imagePullPolicy: IfNotPresent
  ports:
    - containerPort: 8761
  envFrom:
    - configMapRef:
        name: service-registry-config
  readinessProbe:
    httpGet:
      path: /actuator/health
      port: 8761
      initialDelaySeconds: 10
  livenessProbe:
    httpGet:
      path: /actuator/health
      port: 8761
      initialDelaySeconds: 20

---
apiVersion: v1
kind: Service
metadata:
  name: service-registry
  namespace: quiz-app
spec:
  selector:
    app: service-registry
  ports:
    - port: 8761
      targetPort: 8761

---
# API-GATEWAY
apiVersion: v1
kind: ConfigMap

```

```

metadata:
  name: api-gateway-config
  namespace: quiz-app
data:
  SPRING_APPLICATION_NAME: api-gateway
  SERVER_PORT: "8080"
  # Point to config-server and Eureka
  SPRING_CLOUD_CONFIG_URI: http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://service-registry:8761/eureka
  # Ex: SPRING_PROFILES_ACTIVE: prod
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: quiz-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: api-gateway
          image: your-registry/api-gateway:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:

```

```

        name: api-gateway-config
    readinessProbe:
        httpGet:
            path: /actuator/health/readiness
            port: 8080
        initialDelaySeconds: 10
    livenessProbe:
        httpGet:
            path: /actuator/health/liveness
            port: 8080
        initialDelaySeconds: 20
---
apiVersion: v1
kind: Service
metadata:
    name: api-gateway
    namespace: quiz-app
spec:
    type: LoadBalancer # or NodePort if your cluster doesn't support LB
    selector:
        app: api-gateway
    ports:
        - port: 80
          targetPort: 8080
---
# CATEGORY-SERVICE
apiVersion: v1
kind: ConfigMap
metadata:
    name: category-service-config
    namespace: quiz-app
data:
    SPRING_APPLICATION_NAME: category-service

```

```

SERVER_PORT: "8081"
SPRING_CLOUD_CONFIG_URI: http://config-server:8888
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://service-registry:8761/eureka
# Example DB config via env (use Secret for passwords)
SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/categories
SPRING_DATASOURCE_USERNAME: categories_user
# SPRING_DATASOURCE_PASSWORD should come from a Secret

```

```

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: category-service
  namespace: quiz-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: category-service
  template:
    metadata:
      labels:
        app: category-service
    spec:
      containers:
        - name: category-service
          image: your-registry/category-service:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8081
          envFrom:
            - configMapRef:
                name: category-service-config
            # - secretRef:

```

```

        #      name: category-service-secrets
    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
        port: 8081
      initialDelaySeconds: 10
    livenessProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8081
      initialDelaySeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: category-service
  namespace: quiz-app
spec:
  selector:
    app: category-service
  ports:
    - port: 8081
      targetPort: 8081
---
# QUESTIONS-GENERATOR
apiVersion: v1
kind: ConfigMap
metadata:
  name: questions-generator-config
  namespace: quiz-app
data:
  SPRING_APPLICATION_NAME: questions-generator
  SERVER_PORT: "8082"
  SPRING_CLOUD_CONFIG_URI: http://config-server:8888

```

```
EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://service-registry:8761/eureka
```

```
---
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: questions-generator
  namespace: quiz-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: questions-generator
  template:
    metadata:
      labels:
        app: questions-generator
    spec:
      containers:
        - name: questions-generator
          image: your-registry/questions-generator:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8082
          envFrom:
            - configMapRef:
                name: questions-generator-config
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8082
              initialDelaySeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
```

```

        port: 8082
        initialDelaySeconds: 20

---
apiVersion: v1
kind: Service
metadata:
  name: questions-generator
  namespace: quiz-app
spec:
  selector:
    app: questions-generator
  ports:
    - port: 8082
      targetPort: 8082

---
# QUIZ-SERVICE
apiVersion: v1
kind: ConfigMap
metadata:
  name: quiz-service-config
  namespace: quiz-app
data:
  SPRING_APPLICATION_NAME: quiz-service
  SERVER_PORT: "8083"
  SPRING_CLOUD_CONFIG_URI: http://config-server:8888
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://service-registry:8761/eureka

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quiz-service
  namespace: quiz-app

```

```

spec:
  replicas: 2
  selector:
    matchLabels:
      app: quiz-service
  template:
    metadata:
      labels:
        app: quiz-service
    spec:
      containers:
        - name: quiz-service
          image: your-registry/quiz-service:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8083
          envFrom:
            - configMapRef:
                name: quiz-service-config
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8083
              initialDelaySeconds: 10
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8083
              initialDelaySeconds: 20
      ---
apiVersion: v1
kind: Service
metadata:
  name: quiz-service
  namespace: quiz-app

```

```
spec:
  selector:
    app: quiz-service
  ports:
    - port: 8083
      targetPort: 8083
```

Dynamic provisioning

Scale Pods to Handle More Traffic

```
kubectl scale deployment <dep name> --replaces=5
```

Rollout update with zero downtime

```
kubectl set image deployment/web nginx=nginx:1.25
```

```
#check the status
```

```
kubectl rollout status deployment/web
```

HPA

HPA automatically increases or decreases the number of Pods based on metrics like:

- CPU Utilization
- Memory Utilization
- Custom Metrics

- External Metrics (e.g., Kafka queue length, Prometheus metrics)

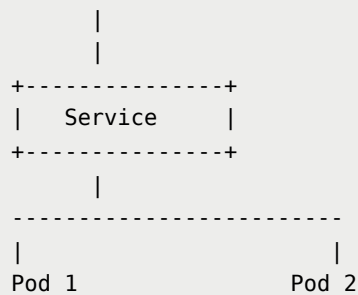
Without HPA

Suppose you deploy:

```
spec:
  replicas: 2
```

You always have:

User Requests



Even if traffic increases from 100 requests/sec to 5000 requests/sec,
you still have only **2 Pods**.

Both Pods become overloaded.

With HPA

Traffic increases.

100 Requests/sec

↓

CPU becomes

85%

HPA notices high CPU.

↓

Creates more Pods.

Pod1
Pod2
Pod3
Pod4
Pod5

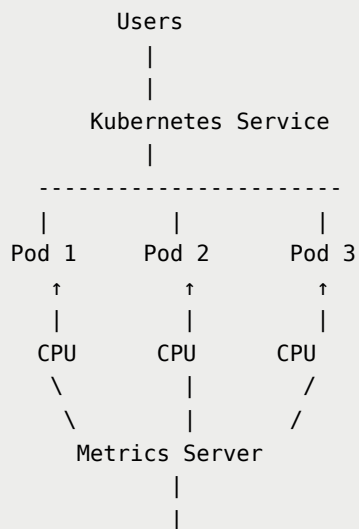
Traffic decreases.

CPU drops to

20%

HPA automatically removes extra Pods.

How HPA Works



Horizontal Pod Autoscaler

|
Increase / Decrease Pods

Prerequisites

Before HPA can work, Kubernetes must know the CPU or memory usage of Pods.

For that, you need:

1. **Metrics Server** installed in the cluster.
2. **CPU/Memory requests** defined in the Deployment.

Without resource requests, HPA cannot calculate utilization percentages.

Step 1: Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: springboot-app

spec:
  replicas: 2

  selector:
    matchLabels:
      app: springboot

  template:
    metadata:
      labels:
        app: springboot

    spec:
      containers:
        - name: springboot
          image: durgesh/springboot:latest

      ports:
        - containerPort: 8080

      resources:
```

```
requests:
  cpu: "200m"
  memory: "256Mi"

limits:
  cpu: "500m"
  memory: "512Mi"
```

Why resources are required?

Suppose

CPU Request = 200m

Current usage becomes

160m

CPU Utilization

```
160 / 200 × 100
= 80%
```

HPA uses this percentage to decide whether to scale.

Step 2: Create HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: springboot-hpa

spec:

  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```
name: springboot-app

minReplicas: 2

maxReplicas: 10

metrics:
- type: Resource

  resource:
    name: cpu

  target:
    type: Utilization
    averageUtilization: 70
```

Understanding Each Property

scaleTargetRef

Which Deployment should be scaled?

```
scaleTargetRef:
  kind: Deployment
  name: springboot-app
```

minReplicas

Minimum Pods.

```
minReplicas: 2
```

Even if there is no traffic,
Pods will never go below 2.

maxReplicas

Maximum Pods.

```
maxReplicas: 10
```

Even if CPU reaches 100%,
Kubernetes will not create more than 10 Pods.

averageUtilization

```
averageUtilization: 70
```

Meaning

Target CPU

```
70%
```

If CPU exceeds 70%,
create more Pods.

If CPU goes well below the target,
remove Pods (but never below `minReplicas`).

Example

Current Pods

```
2
```

Each Pod CPU

```
90%
```

Target

70%

Result

Scale

2 → 3 Pods

Later

CPU

85%

Scale

3 → 4

Later

CPU

72%

Keep current Pods.

Later

CPU

20%

Scale Down

4 → 3

↓

3 → 2

Real Timeline

10:00 AM

Traffic = 100 users

Pods = 2

CPU = 35%

↓

11:00 AM

Traffic = 1000 users

CPU = 90%

↓

HPA

Creates

Pod3

Pod4

Pod5

↓

CPU becomes

55%

Everything is stable.

Scale Down

Night time.

Traffic

20 users

CPU

15%

HPA removes Pods.

5

↓

4

↓

3

↓

2

Commands

Create HPA

```
kubectl apply -f hpa.yaml
```

Check HPA

```
kubectl get hpa
```

Example output

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
springboot-hpa	Deployment/springboot-app	65%/70%	2	10	3

Describe HPA

```
kubectl describe hpa springboot-hpa
```

Watch Pods

```
kubectl get pods -w
```

Watch Deployment

```
kubectl get deployment
```

Metrics Server

Check whether Metrics Server is installed:

```
kubectl top pods
```

or

```
kubectl top nodes
```

If these commands return CPU and memory usage, Metrics Server is working. Without Metrics Server, HPA based on CPU or memory will not function.

Can HPA Scale Based on Traffic?

Not directly.

HPA does **not** measure HTTP requests per second on its own.

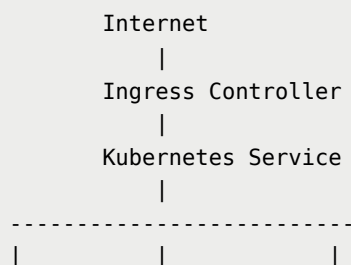
Instead, it scales based on metrics such as:

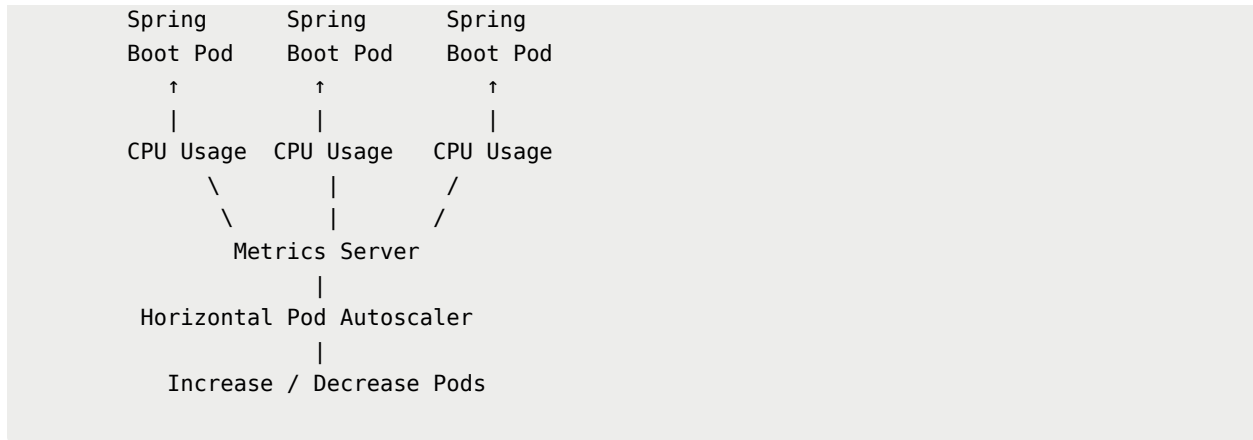
- CPU
- Memory
- Custom Metrics
- External Metrics

If your application receives more traffic, CPU usage usually increases, and HPA reacts to that increase.

If you want to scale based on **requests per second**, **Kafka queue length**, or **Prometheus metrics**, you need custom or external metrics adapters (for example, Prometheus Adapter or KEDA).

Production Architecture





Helm

elm is one of the most important tools in Kubernetes. It is often called the **Package Manager for Kubernetes**, just like:

- **apt** → Ubuntu packages
- **npm** → Node.js packages
- **Maven** → Java libraries
- **Helm** → Kubernetes applications

What Problem Does Helm Solve?

Imagine you have a Spring Boot application.

Without Helm, you may have to create:

- deployment.yaml
- service.yaml
- configmap.yaml
- secret.yaml
- ingress.yaml
- hpa.yaml

- pvc.yaml

That's already **7+ YAML files**.

Now imagine you deploy the same application in:

- Development
- Testing
- Staging
- Production

You'll end up maintaining **20–50 YAML files**.

This becomes difficult.

Helm solves this by allowing you to create **templates**.

Before Helm

```
springboot/  
  deployment-dev.yaml  
  deployment-prod.yaml  
  deployment-test.yaml  
  
  service-dev.yaml  
  service-prod.yaml  
  
  ingress-dev.yaml  
  ingress-prod.yaml  
  
  secret-dev.yaml  
  secret-prod.yaml
```

Lots of duplicate YAML.

With Helm

```
springboot-chart/  
  
  templates/
```

```
deployment.yaml
service.yaml
ingress.yaml
secret.yaml

values.yaml
```

One template.

Different values.

What is Helm?

Helm packages Kubernetes applications into something called a **Chart**.

Think of a chart as a ZIP file containing everything needed to deploy an application.

```
Chart
|
|---- Deployment
|---- Service
|---- ConfigMap
|---- Secret
|---- PVC
|---- Ingress
```

One command installs everything.

```
helm install myapp .
```

What is a Helm Chart?

A Helm Chart is simply a folder.

Example:

```
mychart/
```

```
Chart.yaml

values.yaml

templates/

  deployment.yaml
  service.yaml
  ingress.yaml
  configmap.yaml
```

Every chart has this structure.

Chart.yaml

This contains metadata.

```
apiVersion: v2
name: springboot-app
description: Spring Boot Application
type: application
version: 1.0.0
appVersion: "1.0"
```

Meaning

```
name
↓

springboot-app
```

```
version
↓

Chart version
```

```
appVersion
↓
```

values.yaml

This is the most important file.

Instead of hardcoding values...

Bad

```
replicas: 3

image:
  repository: durgesh/springboot
  tag: v5

service:
  port: 8080
```

These values can change without touching the templates.

templates/

Templates contain Kubernetes YAML files.

Instead of writing

```
replicas: 3
```

Helm writes

```
replicas: {{ .Values.replicas }}
```

Instead of

```
image: durgesh/springboot:v5
```

Template

```
image:"{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Now values come from

```
values.yaml
```

Example

values.yaml

```
replicas: 2

image:
  repository: durgesh/springboot
  tag: latest
```

deployment.yaml

```
spec:
  replicas: {{ .Values.replicas }}

  containers:
  - image:"{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Helm generates

```
spec:
  replicas: 2
```

```
containers:
- image: durgesh/springboot:latest
```

How Templates Work

```
Template

{{ .Values.replicas }}

      ↓

Values.yaml

replicas: 5

      ↓

Generated YAML

replicas: 5
```

Installing Helm

Mac

```
brew install helm
```

Ubuntu

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

Verify

```
helm version
```

Creating a Chart

```
helm create springboot-chart
```

Output

```
springboot-chart
  Chart.yaml
  charts/
  templates/
  values.yaml
```

Install Chart

Inside chart

```
helm install myapp .
```

or

```
helm install myapp springboot-chart/
```

Check Release

```
helm list
```

Output

```
NAME
myapp
```

Upgrade

Suppose you changed image tag.

```
latest
↓
v2
```

Run

```
helm upgrade myapp .
```

Everything updates automatically.

Rollback

One of Helm's biggest advantages.

Suppose deployment fails.

```
Version 1
↓
Version 2
↓
Crash
```

Rollback

```
helm rollback myapp1
```

Application returns to the previous version.

Uninstall

```
helm uninstall myapp
```

Everything created by Helm gets deleted.

Common Helm Commands

Command	Purpose
<code>helm create mychart</code>	Create a chart
<code>helm install myapp .</code>	Install chart
<code>helm list</code>	List releases
<code>helm status myapp</code>	Show release status
<code>helm upgrade myapp .</code>	Upgrade release
<code>helm rollback myapp 1</code>	Roll back to revision 1
<code>helm uninstall myapp</code>	Remove release
<code>helm template .</code>	Render templates locally without deploying
<code>helm lint .</code>	Validate the chart

Example Project Structure

```
springboot-chart/  
|  
├─ Chart.yaml  
|
```

```
├── values.yaml
└── templates
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── configmap.yaml
    ├── secret.yaml
    ├── hpa.yaml
    └── pvc.yaml
```

Real-World Example

Suppose your Spring Boot application has:

- Docker Image
- MySQL
- ConfigMap
- Secret
- Ingress
- PVC
- HPA

Without Helm:

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml
kubectl apply -f pvc.yaml
kubectl apply -f configmap.yaml
kubectl apply -f secret.yaml
kubectl apply -f hpa.yaml
```

With Helm:

```
helm install blog-app .
```

That's it.

Advantages of Helm

- Reusable templates
- Environment-specific configurations (dev, test, prod)
- Easy upgrades
- One-command deployments
- Versioning of releases
- Rollbacks
- Parameterized YAML using `values.yaml`
- Easy sharing through Helm repositories
- Widely used in production Kubernetes environments